See the matrix through its factors

A practical guide to the WPF application, its mathematics, and its current limits.

SvdStudio makes $\mathbf{A = U \Sigma V^T}$ tangible: inspect the factors, follow the spectrum, reduce rank, and see the effect on a matrix or grayscale image. It is a Windows application with a separate managed C# numerical core.

Start from the repository root

Install the .NET 8 SDK on Windows. The repository includes the WPF project, core library, and console verifier as sibling folders. Keep them together.

```
dotnet restore SvdStudio.sln
dotnet build SvdStudio.sln -c Release --no-restore
dotnet run --project SvdStudio/SvdStudio.csproj -c Release --no-build
```

A two-minute exploration

Enter **[3 0; 0 1]** and compute the decomposition. Find singular values 3 and 1, numerical rank 2, and determinant magnitude 3. Switch to the low-rank tab and set k to 1: the result is diag(3,0), with absolute Frobenius residual 1.

Open the 2D geometry tab. The circle becomes an ellipse with semi-axis lengths 3 and 1. In the image studio, move the rank slider and compare image detail with the number of factor values retained.

How to use this guide

Pages 2-4 explain the workspace and mathematics. Pages 5-6 describe implementation, architecture, and verification. Page 7 covers public sharing and references. The editable Markdown companion and README contain portable commands and links.

Prepared November 3, 2026. MIT license selected by the project owner. Verification: 251 assertions passed, 0 failed. GUI interaction testing remains a manual release step.

Use the workspace

From pasted numbers to factors, diagnostics, and approximations.

Tab / tool	Purpose
Factorization	Inspect A, U, Σ, V, V^T, and the reconstructed matrix using heatmaps. Choose economy or full factors.
Spectral / scree	Read singular values, per-mode squared energy, cumulative retained energy, and the pseudoinverse.
Low-rank approximation	Choose k, inspect the approximation and residual, and isolate individual rank-1 contributions.
2D geometric action	For exactly 2×2 input, inspect the circle-to-ellipse map and singular directions.
Image studio	Choose a procedural pattern or local image, then compare grayscale low-rank reconstructions.
Theory reference	Read a short in-app explanation of SVD, approximation, and pseudoinverses.

Input and output

Paste numbers separated by spaces, tabs, or commas, with one row per line. MATLAB-style input uses semicolons between rows. Numeric nested arrays are also accepted. Use equal row lengths and decimal points. Text import, presets, and random generation provide other entry points.

```
1 2 3
4 5 6

[1 2 3; 4 5 6]

[[1, 2, 3], [4, 5, 6]]
```

Matrix commands copy CSV, LaTeX, MATLAB, or JSON. The summary export writes a formatted text report. The parser is permissive; it is not a general JSON parser or a complete quoted-CSV implementation.

Read the diagnostics

Rank and condition estimates depend on the numerical threshold. Reconstruction and orthogonality errors describe the computed factors. Display formatting rounds values; use diagnostics rather than rounded cells to judge accuracy. Computations are synchronous, so large inputs can pause the UI.

Understand the factors

Orthogonal directions, non-negative scales, and the four subspaces.

For a real m×n matrix, let p = min(m,n). In full SVD, U and V are square orthogonal matrices and Σ is rectangular diagonal. Its p diagonal values are non-negative and ordered from largest to smallest. [1]

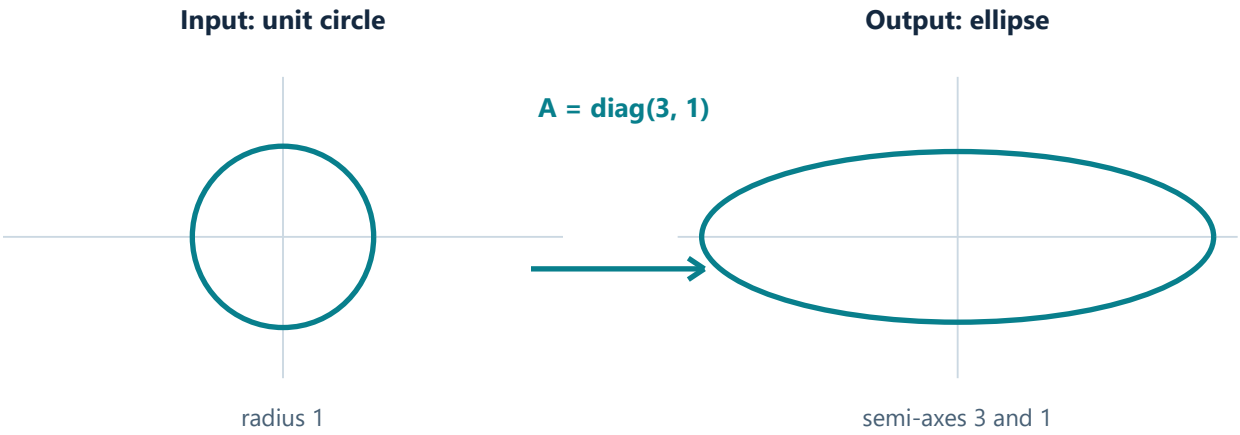
```
A = U Σ V^T
σ1 >= σ2 >= ... >= σp >= 0
A vi = σi ui
```

Mode	U	Σ	V^T
Economy	m×p	p×p	p×n
Full	m×m	m×n	n×n

Economy mode retains p directions, including zero-valued modes. It is not the rank-r compact decomposition. With rank r, the first r columns of U span the column space and the first r columns of V span the row space. In full mode, the remaining m-r columns of U span $N(A^T)$, and the remaining n-r columns of V span N(A).

Geometry in two dimensions

The action V^T changes input coordinates, Σ scales them, and U changes output coordinates. Orthogonal transformations can include reflections as well as rotations. Singular values are ellipse semi-axis lengths; their product is the determinant magnitude for a 2×2 matrix.



A zero singular value collapses a direction. Singular vectors can differ in sign between valid decompositions, and repeated singular values allow different orthonormal bases in their shared subspace. Compare reconstructed matrices and subspaces rather than individual signs.

Keep the strongest modes

Low-rank approximation, energy, pseudoinverses, and image experiments.

```
Ak = sum(i=1..k) σi ui vi^T
||A - Ak||F = sqrt(sum(i=k+1..p) σi²)
||A - Ak||2 = σ(k+1), or 0 when k = p
```

The Eckart-Young-Mirsky result gives an optimal approximation among matrices of rank at most k in the Frobenius and spectral norms. The omitted singular values determine the residual. A minimizer need not be unique when a truncation boundary has repeated singular values. [3]

Squared singular-value energy: 9 + 1 = 10



Rank 1 retains 90% 10% discarded

Frobenius residual = 1; relative residual = 1 / sqrt(10) = 31.6%

Retained energy is the fraction of the sum of squared singular values kept. The residual percentage is based on a norm, so it is not 100% minus retained energy. Statistical “variance explained” requires the appropriate centered-data/PCA interpretation.

Pseudoinverse

```
A+ = sum(σi > tau) (1 / σi) vi ui^T
```

The exact Moore-Penrose inverse gives a minimum-norm least-squares solution $x = A^{+} b$. The implementation drops singular values at or below its threshold. When small nonzero values are dropped, this is a truncated approximation to the exact pseudoinverse.

Image approximation and storage

Procedural patterns are 48×48. File import requests a decode width of 64 pixels, then converts to 8-bit grayscale. The display compares k(m+n+1) factor values with mn original values. It does not compare encoded PNG/JPEG bytes or account for floating-point byte widths and metadata. High ranks can require more values than the original. The studio previews results; it does not save a compressed image file.

Know the numerical limits

What the implementation computes, and where interpretation matters.

Algorithm path

Householder reflections reduce tall or square input to upper bidiagonal form. The engine accumulates left and right transformations, then uses implicit shifted QR steps and deflation to obtain ordered, non-negative singular values. Wide matrices are decomposed through their transpose and mapped back. This is a managed Golub-Kahan-Reinsch-style implementation, not a call to LAPACK.

Default threshold - actual code behavior

```
eps = 2.2204460492503131e-16
tau = max(m,n) * eps * max(σ1, 1)
rank = count(σi > tau)
```

The max(σ1,1) factor imposes an absolute floor. A matrix such as 1e-20 times the identity can have computed nonzero singular values but default numerical rank zero. The core API accepts a finite, non-negative customTolerance. The UI uses the default. The same threshold is used for pseudoinverse truncation and the finite/infinite condition estimate.

Diagnostics and guarantees

The result reports absolute reconstruction residual $||A-U\Sigma V^T||F$, maximum-entry residual, and orthogonality residuals $||U^TU-I||F$ and $||V^TV-I||F$. Relative residuals are more informative when comparing matrices at different scales. Standard numerical analysis relates errors to matrix scale, dimensions, and floating-point precision; it does not justify a universal absolute error promise. [2]

Finite input is required. NaN and infinity are rejected, and an exhausted iteration limit raises an exception. Very large or very small finite values may still overflow or underflow intermediate arithmetic. Energy and relative-error displays use absolute small-value cutoffs and can return conventional 100% or zero values for tiny inputs.

Useful invariants

The spectral norm is σ1, the Frobenius norm is the square root of the squared singular-value sum, and the nuclear norm is their sum. For square matrices, the singular-value product gives |det(A)|, not its sign. The displayed compute time excludes result diagnostics and UI preparation.

Inspect the engineering

MVVM structure, verification evidence, and preparation changes.

Layer	Responsibilities
Core model	Matrix storage and operations, input parsing, presets, SVD, approximation, and pseudoinverse. No WPF dependency.
View models	Input/result state, commands, formatted cells, geometry data, image conversion, and tab coordination.
Views / controls	XAML bindings, heatmap rendering, canvas plots, and layout-related code-behind.

MainWindow code-behind initializes the view and assigns MainViewModel. ObservableObject supplies property notifications and RelayCommand supplies commands. The view models also use WPF types and dialogs, so the presentation layer is not platform-independent. There is no dependency-injection composition root or asynchronous computation pipeline.

What was verified

The original console suite passed 114 assertions. The prepared suite passes **251 assertions with 0 failures**. It checks reconstruction, orthogonality, dimensions, ordering, rank, parsing examples, approximation behavior, and pseudoinverse identities. Most residual checks use absolute tolerance 1e-10; the test count is assertions, not independent test methods.

New coverage includes zero matrices, single rows and columns, invalid solver inputs/tolerances, all four Penrose identities on rank-deficient rectangular examples, and the theoretical discarded-tail Frobenius residual. The source review and successful build do not replace a GUI smoke test or independent numerical-library comparison.

Changes in the prepared copy

The solver now rejects non-finite entries and invalid custom tolerances, and throws when QR reaches its iteration limit. The geometry tab is limited to exactly 2×2 matrices; the former larger-matrix view mixed incompatible projections. Theory wording and report-export labeling were corrected. The repository adds a solution, MIT license, SDK selection, ignore rules, CI workflow, and these documents.

No GUI interaction test, performance benchmark, signed installer, or hosted CI run was completed during preparation. Follow docs/Validation.md before publishing a downloadable binary. The original source directories remain unchanged.

Share the project

A reviewable source package with a separate publication step.

Repository and release workflow

Use the common root containing README.md, SvdStudio.sln, and all three project folders. Review the MIT license and validation notes, then build and run the console verifier. The included Windows workflow repeats restore, build, and verification on pushes and pull requests.

Initialize Git in that root, stage the files, review the staged list, and commit. With authenticated GitHub CLI, create a public repository from the committed source. The publication command and an alternative website-based route are in docs/GitHub-Publishing.md. No remote repository was created during this preparation.

After the first push, confirm the Actions run passes and README/PDF links work. For a binary release, first complete the manual GUI checklist. A framework-dependent publish requires the .NET 8 Desktop Runtime; keep all published files together and attach the ZIP to a release rather than committing build outputs.

References

[1] LAPACK Users' Guide: Singular Value Decomposition

[2] LAPACK Users' Guide: Further Details on SVD Error Bounds

[3] Ankur Moitra, MIT course notes, Theorem 2.1.2

[4] JAMA: NIST / The MathWorks public-domain reference

[5] GitHub: Adding locally hosted code

Provenance note

The solver structure resembles JAMA's four-case QR implementation; this is an inference, not a confirmed derivation history. The inspected files do not establish complete provenance. No reference-library source was imported during preparation. The owner selected MIT for this package. Further details and full reference links appear in docs/References.md.

Documentation describes the prepared copy as of November 3, 2026. Read docs/Validation.md and verification-summary.txt for the precise build and verification record.